

S-38.3159: Protocol Diary

Hämäläinen, Harri	Islam, Hasan
<code>harri.hamalainen@aalto.fi</code>	<code>hasan.islam@aalto.fi</code>

Kurnikov, Arseny
`arseny.kurnikov@aalto.fi`

January 27, 2016

Protocol Diary

Design Consideration

Implementation started with some open questions regarding for design considerations such as follows:

- Reliable transport?
- Protocol Encoding?
- Persistence?
- Message types?

Sensor code uses UDP without any guarantee of reliability. We decided to include reliability as a service which can be turned on with parameter given in SUBSCRIBE message. If the reliability parameter is turned on the server must listen for acknowledgement from subscriber with the timestamp of the last message.

JSON used on communications between server and sensors in the code provided to us. Camera sends binary data, others can be either encoded or sent as text. We decided to implement subset of BSON parser for easier parsing at clients.

The discussion for persistence focused on what to do with dead clients in order to minimize the useless communication with those. SUBSCRIBE message must be refreshed periodically (this can be seen as keep-alive messaging from client) or as a separate option e.g., one is keep-alive, the other one is (RE)SUBSCRIBE.

We discussed whether we pack all updates in one message or have separate port for "streaming" data and acknowledgements for more sensitive one. For keep-alive message, if no keep-alive message is received within 2 keep-alive intervals, the client will be dropped.

Performance issues

When a client is silently dropped we can go through all the recipient lists and look for that client; or we can store pointers to lists where that client is currently present and deal only with those. We considered this issue based on implementation question; how we actually serve: multi-threaded, fork, single server. We considered that good structuring should be sufficient to

have reasonable performance.

We also discussed whether we do multiplex everything on one port or use different ports for different clients/sensors. We come up with to use two ports, one for sensors and the other for clients.

At the very beginning, we did not bother about fragmentation too much. The messages from the sensors except camera are quite small. Our design at that moment assumed that the messages are sent and received as is and no fragmentation can happen in higher than L3 layer.

BSON Encoder

BSON encoder implementation considered to have requirement for fixed order for keys (i.e. message type must be the first key) and callbacks to process the rest of the message (after the correct callback is inferred from the message type). The other option (the generic one) is to pass the BSON object parsed by BSON to request handler which will do the inferring of required further processing. We attributed it as cleaner solution, but it might create quite a bit overhead.

Packet loss generator

We decided to implement the markovian error / packet loss generator and add into the project. The idea of two state markovian packet loss generator is that you just drop the packet to be sent with probability p and send it with probability of $1-p$ when no packet is lost. When a packet is lost the next packet is also lost i.e. dropped with probability of q and it's send with probability of $1-q$.

Congestion Control

To summarize our design ideas from the first specification and to go through some ideas for the congestion control mechanism:

- We only provide a method to guarantee the delivery of the most recent message when the subscription is created with reliable flag. This already is a very primitive congestion control mechanism.
- We can get information about the condition of the network from losses and delays, we cannot use queue size as an indicator as we use UDP
- Delay based congestion control

- We have no way to get an idea about the real RTT for delay measurements (all messages except the update/update ack pair are in one direction only with no replies).
- The delay based congestion control is still quite a problematic to do right.
- Adding keep-alive acks from server would create communication pattern with fixed size suitable for measurements, but the pattern is client→server→client, but we require also measurements from server→client→server.
- It's more critical to have a good measurements from the server point of view, as most of the traffic is created by the servers
- Loss based congestion control
 - For example the loss of keep-alive or update-ack most likely indicates a congestion
 - Can be false positive if the messages are sent and received via different paths
 - If a loss is detected then the idea is where we try to detect the congestion and still provide service to clients, but at the same time the messages we deliver to clients indicate that the network is congested, so that the client can take actions (and for example tear down connections or fix the network) or skipping the delivery of new sensor updates in order not to create more congestions.

We decided to do loss-based congestion control on the server side. The client will notice the packet loss from the sequence numbers. Sending NACKs for unreliable transmission doesn't look too attractive. Maybe we can just rely on reliable transmissions and ACKs from them. Of course then unreliable congestion will be an issue. Throttling down will be difficult to implement. Suddenly we have to manage bigger buffers for sensor updates, pointers in those buffers for each subscription, handling rates. Skipping an update is rather simple: if congested flag is set, increase seq_no, but don't send anything.

The question we focused on how to detect lost messages when we can use only KEEP_ALIVE messages and UPDATE_ACKs. One possibility to inform the server about the losses is to send "NACK style" or UPDATE_ACK message (even if the connection is set up to be Unreliable) when a new update is received with greater sequence number then the next expected sequence number. However this would really add more messages into congested network. Additionally, the latency of this would probably be too huge.